

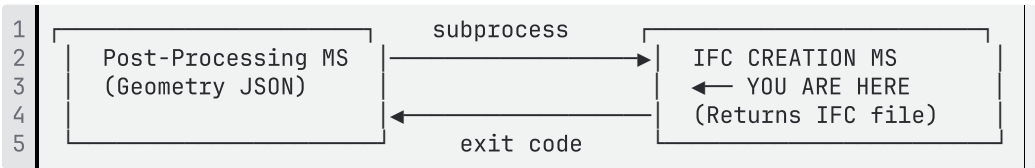
IFC Creation Microservice Architecture and Function Flow

The IFC Creation Microservice is the final model-generation stage in the Inshelter Scan2BIM pipeline. It receives structured geometry JSON from the Post-Processing Microservice, converts that data into an IFC 2x3 BIM model using the xBIM Toolkit, and writes the resulting `.ifc` file back to disk for the caller to upload and signal onward.

Scope of this page: This page documents the IFC Creation subprocess only. It starts when Post-Processing invokes `dotnet IFCCreator.dll <json_path>` and ends when the IFC file is written and an exit code is returned.

- [Position in the pipeline](#)
- [Service overview](#)
- [What this subprocess is responsible for](#)
- [High-level architecture](#)
- [Module structure](#)
- [End-to-end function flow](#)
- [Entry point behavior](#)
- [Core builder flow](#)
- [Data model used by the subprocess](#)
- [IFC entity mapping](#)
- [IFC hierarchy produced](#)
- [Element creation sequence](#)
- [Invocation from Post-Processing](#)
- [Input and output contract](#)
- [Connection to the previous and next stages](#)
- [Failure handling and operational expectations](#)
- [Operational summary](#)

Position in the pipeline



Direction	Service	Interaction
Upstream	Post-Processing MS	Invokes the .NET console app through <code>dotnet IFCCreator.dll</code>

		<code><json_path></code> and waits for completion
Downstream	None	No direct downstream service call; the subprocess returns an IFC artifact and exit status to the caller

Service overview

This component is intentionally narrow in scope. It does not extract geometry from point clouds, run machine-learning inference, upload files, or signal workflow completion. Its responsibility is to transform an already structured intermediate JSON model into a standards-based IFC file with the expected project, site, building, storey, element geometry, and attached property sets.

Property	Value
Primary language	C# (.NET)
Runtime mode	Console application invoked as a subprocess
IFC schema	IFC 2x3
Core library	xBIM Toolkit
Invocation	<code>dotnet IFCCreator.dll</code> <code><json_path></code>
Timeout behavior	5-minute timeout enforced by the calling Post-Processing service
Input	Structured geometry JSON serialized by Post-Processing
Output	<code>New_{filename}.ifc</code> in the same directory as the input JSON

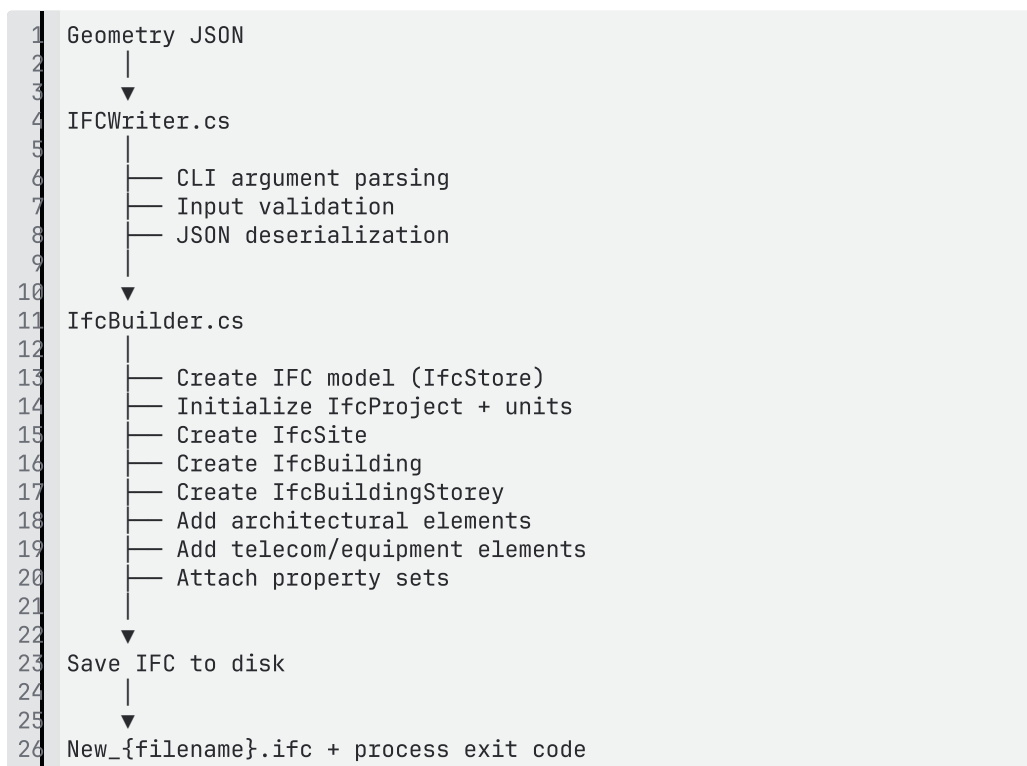
Why this subprocess exists: The pipeline separates geometry interpretation from BIM serialization. Python-based Post-Processing prepares the semantic object graph, while the .NET subprocess specializes in standards-compliant IFC writing.

What this subprocess is responsible for

- Parse CLI arguments and validate the input JSON path
- Deserialize JSON into strongly typed C# model classes
- Create an in-memory IFC model using IFC 2x3 entities
- Initialize project structure, including project, site, building, and building storey
- Translate each JSON component into the correct IFC entity type and shape
- Create parent-child and spatial containment relationships between elements
- Attach project, site, and element property sets where provided
- Persist the completed IFC model to disk
- Return a process exit code that the caller can use for success/failure handling

Out of scope: File upload, queue handling, retry orchestration, Temporal signaling, and blob backup all belong to the Post-Processing Microservice, not this subprocess.

High-level architecture



Module structure

```
1 IFCCreator/  
2 |— IFCWriter.cs # Entry point (Main): CLI argument  
parsing  
3 |— IFCCreator.csproj # .NET project file  
4 |— Services/  
5 |— IfcBuilder.cs # Core model creation and element  
writing  
6 |— IfcPropertySet.cs # Property set creation and  
attachment  
7 |— IfcApiIntegrationService.cs # HTTP file download utility  
8 |— Models/  
9 |— CommonData.cs # BuildingStorey, Cuboid, Point3D,  
Opening  
10 |— ArchitecturalElements.cs # Wall, Door, FloorCeiling  
11 |— TelecomEquipments.cs # Cabinet, BatteryCabinet, Rack,  
Battery, Rectifier  
12 |— Utils/  
13 |— JsonReader.cs # Deserializes JSON to  
BuildingStorey  
14 |— IfcBuilderHelpers.cs # Geometry helper methods  
15 |— Validation.cs # Input file validation  
16 |— Scripts/  
17 |— Test-Files/ # Sample JSON inputs
```

End-to-end function flow

The runtime path is short but important: validate input, deserialize data, build the IFC model, write the file, and return a process code.

```
1 Main(args)  
2 → validate CLI input  
3 → validate JSON path  
4 → deserialize JSON into BuildingStorey  
5 → compute destination IFC path  
6 → invoke IfcBuilder.CreateIfcFile(...)  
7 → return 0 on success / 1 on error
```

Stage	Purpose	Outcome
Argument parsing	Read the JSON file path passed by Post-Processing	Valid runtime input for the subprocess
Validation	Ensure the JSON file exists and is readable	Early failure if the handoff artifact is missing
Deserialization	Convert JSON to strongly typed domain classes	In-memory BuildingStorey object graph

IFC model creation	Build IFC entities and relationships from the object graph	Complete in-memory IFC 2x3 model
Persistence	Save the generated file to disk	New_{filename}.ifc
Process result	Communicate success or failure to the caller	Exit code and console output

Entry point behavior

```

1 static int Main(string[] args)
2 {
3     // 1. Parse CLI argument: json_path
4     // 2. Validate JSON file exists
5     // 3. Deserialize JSON → BuildingStorey model
6     // 4. Create IFC file via IfcBuilder
7     // 5. Return exit code (0=success, 1=error)
8 }

```

The entry point acts as the subprocess contract boundary. It is intentionally minimal so that Post-Processing can treat the IFC engine as a deterministic command-line tool.

Core builder flow

```

1 public void CreateIfcFile(BuildingStorey geometryData, string
destinationPath)
2 {
3     // 1. CreateAndInitModel(projectName)
4     //   → IfcStore (IFC 2x3, in-memory)
5     //   → IfcProject with SI units
6
7     // 2. CreateIfcSite(model, "Default", geometryData)
8
9     // 3. CreateIfcBuilding(model, "Default")
10
11    // 4. CreateIfcBuildingStorey(model, geometryData)
12    //   → The main container for all elements
13
14    // 5. Add elements to storey:
15    //   a) AddWalls() → IfcWall
16    //   b) AddDoors() → IfcDoor
17    //   c) AddFloorCeilings() → IfcSlab
18    //   d) AddCabinets() → IfcDistributionElement
19    //   e) AddBatteryCabinets() → IfcDistributionElement
20    //   f) AddRacks() → IfcFurniture
21    //   g) AddBatteries() → IfcDistributionElement
22    //   h) AddRectifiers() → IfcDistributionElement
23
24    // 6. SaveIfcFile(model, destinationPath)
25 }

```

The builder is where JSON semantics become BIM semantics. It establishes both geometry and structure: what an object is, where it sits in the spatial hierarchy, and how it relates to its parent container.

Data model used by the subprocess

```
1 public class BuildingStorey
2 {
3     public List<Wall> Wall { get; set; }
4     public List<Door> Door { get; set; }
5     public List<FloorCeiling> Floor_ceiling { get; set; }
6     public List<Cabinet> Cabinet { get; set; }
7     public List<BatteryCabinet> Battery_cabinet { get; set; }
8     public List<Rack> Rack { get; set; }
9     public List<Battery> Battery { get; set; }
10    public List<Rectifier> Rectifier { get; set; }
11    public PropertySet Project_properties { get; set; }
12    public List<PropertySet> Site_properties { get; set; }
13 }
14
15 public class Cuboid
16 {
17     public string Type { get; set; }
18     public double Height { get; set; }
19     public List<Point3D> BottomFacePoints { get; set; }
20     public List<PropertySet> PropertySets { get; set; }
21 }
```

The JSON contract is already normalized by Post-Processing. That means the IFC subprocess expects semantically meaningful collections such as walls, doors, racks, cabinets, batteries, and rectifiers, rather than raw point-cloud data.

IFC entity mapping

JSON component	IFC entity	Geometry strategy
Wall	IfcWall	Cuboid built from four bottom-face points and height
Door	IfcDoor	Opening positioned relative to the parent wall
Floor/Ceiling	IfcSlab	Polygonal slab at the required elevation
Cabinet	IfcDistributionElement	Cuboid enclosure

BatteryCabinet	IfcDistributionElement	Cuboid enclosure with closed sections
Rack	IfcFurniture	Cuboid frame with pillar structure and child equipment
Battery	IfcDistributionElement	Cuboid attached to a parent cabinet context
Rectifier	IfcDistributionElement	Cuboid attached to a parent cabinet context
Rack equipment	IfcDistributionElement	Cuboid child element within a rack

- ✓ **Design intent:** The mapping keeps the IFC output practical rather than overly specialized. Architectural envelope objects use canonical building entities, while telecom equipment is represented using broadly suitable distribution or furniture entities.

IFC hierarchy produced

```

1 IfcProject
2   └─ IfcSite ("Default")
3     └─ IfcBuilding ("Default")
4       └─ IfcBuildingStorey
5         └─ IfcWall (W0, W1, ...)
6           └─ IfcDoor (D0, D1, ...)
7         └─ IfcSlab (Floor, Ceiling)
8         └─ IfcDistributionElement (Cabinet_0, ...)
9         └─ IfcDistributionElement (BatteryCabinet_0, ...)
10            └─ IfcDistributionElement (Battery_0)
11              └─ IfcDistributionElement (Rectifier_0)
12            └─ IfcFurniture (Rack_0, ...)
13              └─ IfcDistributionElement
14                (Rack_equipment_0)
15                └─ IfcDistributionElement
16                (Surge_Protector_0)

```

This hierarchy matters because IFC is not only a geometry container. Spatial decomposition and parent-child relationships determine how authoring tools and viewers interpret the model.

Element creation sequence

1. Create the IFC store and initialize schema context
2. Create `IfcProject` and configure SI units
3. Create the default `IfcSite`

4. Create the default `IfcBuilding`
5. Create the target `IfcBuildingStorey`
6. Add architectural envelope elements first: walls, doors, floor, ceiling
7. Add room equipment and telecom objects: cabinets, battery cabinets, racks
8. Add nested equipment: batteries, rectifiers, rack internals
9. Attach available property sets
10. Persist the model to the output IFC file

Architectural elements are created before dependent objects because later placement often assumes that the enclosing spatial context already exists.

Invocation from Post-Processing

```
1 class IFCCreator:
2     def create_ifc(self, json_file_path: str) -> tuple[bool, str]:
3         process = subprocess.Popen(
4             ["dotnet", str(self.dll_path), json_file_path],
5             stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True
6         )
7         stdout, stderr = process.communicate(timeout=300)
8         return (process.returncode == 0, stdout or stderr)
```

The Python wrapper treats the IFC engine as a black-box executable. Success is based on the subprocess exit code, while stdout or stderr is used for diagnostics.

Input and output contract

Contract area	Expectation
Input file	A valid JSON file on local disk, already written by Post-Processing
Input schema	Structured geometry and property data matching the expected C# model classes
Execution style	Single synchronous CLI invocation per JSON file
Success signal	Process exit code <code>0</code>
Failure signal	Non-zero exit code, typically <code>1</code> , plus console diagnostics

Output file	New_{filename}.ifc in the same directory as the JSON input
-------------	--

Connection to the previous and next stages

Connection to Post-Processing MS

- Post-Processing writes the structured geometry JSON to disk
- It invokes this subprocess with the JSON path
- It waits for completion with a 5-minute timeout
- It reads the exit code and process output
- On success, it expects to find the generated IFC file in the same directory

Connection to the remainder of the pipeline

- This subprocess is the final model-generation step
- It does not upload or distribute the IFC directly
- Post-Processing remains responsible for DataRepo upload, backup upload, and completion signaling to Root MS

Failure handling and operational expectations

Scenario	Expected behavior
Missing CLI argument	Fail fast and return a non-zero exit code
JSON path does not exist	Fail validation before model creation begins
Invalid or malformed JSON	Fail during deserialization and return an error
Incomplete geometry data	Element creation may fail or produce a partial model depending on validation and builder assumptions
Unhandled builder exception	Abort file generation and return an error exit code

Long-running generation	Caller enforces the 5-minute timeout and terminates the subprocess if needed
-------------------------	--

⚠ Critical integration risk: This subprocess assumes that the JSON contract is structurally valid and geometrically coherent. If Post-Processing emits inconsistent parent references, malformed polygons, or missing required dimensions, IFC creation can fail even though upstream extraction completed.

Operational summary

The IFC Creation Microservice is a focused serialization engine. It receives already interpreted geometry from Post-Processing, constructs the IFC 2x3 project hierarchy, maps each object category to an appropriate IFC entity, writes the BIM file to disk, and returns a machine-readable success or failure result to the caller. In the broader pipeline, it is the handoff point where structured geometric understanding becomes a deliverable BIM artifact.